

Graph Theory: A Complete Guide to Shortest Path Algorithms

Theory, Intuition, Proofs and C++ Implementation

Prepared for Deep Study of Shortest Path Algorithms

July 16, 2026

Contents

1	Introduction to Graph Theory and the Shortest Path Problem	3
1.1	1.1 What is a Graph?	3
1.1.1	1.1.1 Directed vs Undirected Graphs	3
1.1.2	1.1.2 Weighted vs Unweighted Graphs	3
1.1.3	1.1.3 Cyclic vs Acyclic Graphs	4
1.2	1.2 What Does “Shortest Path” Mean?	4
1.3	1.3 The Optimal Substructure Property	4
1.4	1.4 The Concept of “Relaxation”	5
1.5	1.5 Graph Representations	5
1.5.1	1.5.1 Adjacency Matrix	5
1.5.2	1.5.2 Adjacency List	6
1.5.3	1.5.3 Edge List	6
1.6	1.6 Notation Used Throughout This Guide	6
2	Breadth-First Search: Shortest Paths in Unweighted Graphs	7
2.1	2.1 Motivation	7
2.2	2.2 Core Idea	7
2.3	2.3 Why BFS Guarantees Shortest Paths	7
2.4	2.4 Multi-Source BFS	8
2.5	2.5 0-1 BFS: A Bridge to Weighted Graphs	8
2.6	2.6 Complexity of BFS	8
2.7	2.7 C++ Implementation	9
3	Dijkstra’s Algorithm	11
3.1	3.1 Motivation	11
3.2	3.2 Core Idea	11
3.3	3.3 Step-by-Step Procedure	11
3.4	3.4 Why Greedy Works: Proof of Correctness	12
3.5	3.5 Data Structure Choice and Complexity	12
3.6	3.6 Path Reconstruction	13
3.7	3.7 Handling Negative Weights: What Goes Wrong	13
3.8	3.8 Complexity Summary	13
3.9	3.9 C++ Implementation	13
4	The Bellman-Ford Algorithm	15
4.1	4.1 Motivation	15

4.2	4.2 Core Idea	15
4.3	4.3 Step-by-Step Procedure	15
4.4	4.4 Why $V - 1$ Iterations Suffice: Proof of Correctness	16
4.5	4.5 Negative Cycle Detection	16
4.6	4.6 SPFA: An Optimization (with a Caveat)	16
4.7	4.7 Complexity Summary	17
4.8	4.8 C++ Implementation	17
5	Shortest Paths in Directed Acyclic Graphs (DAGs)	19
5.1	5.1 Motivation	19
5.2	5.2 Core Idea: Topological Order	19
5.3	5.3 Step-by-Step Procedure	19
5.4	5.4 Proof of Correctness	20
5.5	5.5 Why This Handles Negative Weights Just Fine	20
5.6	5.6 Applications	20
5.7	5.7 Complexity Summary	21
5.8	5.8 C++ Implementation	21
6	Floyd-Warshall: All-Pairs Shortest Paths	22
6.1	6.1 Motivation	22
6.2	6.2 Core Idea: Intermediate Vertices	22
6.3	6.3 The Remarkable In-Place Simplification	23
6.4	6.4 Step-by-Step Procedure	23
6.5	6.5 Negative Cycle Detection	23
6.6	6.6 Complexity Summary	23
6.7	6.7 Johnson's Algorithm: A Faster Alternative for Sparse Graphs with Negative Edges	23
6.7.1	6.7.1 The Reweighting Trick	24
6.7.2	6.7.2 Complexity of Johnson's Algorithm	24
6.8	6.8 C++ Implementation of Floyd-Warshall	24
7	The A* Algorithm: Informed Search	26
7.1	7.1 Motivation	26
7.2	7.2 Core Idea: Heuristic-Guided Priority	26
7.3	7.3 The Admissibility Condition	27
7.4	7.4 Why Admissibility Guarantees Correctness	27
7.5	7.5 Consistency (Monotonicity): A Stronger and More Practical Condition	27
7.6	7.6 A* vs Dijkstra: A Summary Comparison	28
7.7	7.7 Applications	28
7.8	7.8 Complexity Summary	28
7.9	7.9 C++ Implementation (Grid Example with Manhattan Heuristic) . .	28
8	Comparative Summary and Decision Guide	30
8.1	8.1 Choosing the Right Algorithm	30
8.2	8.2 Master Comparison Table	30
8.3	8.3 Common Pitfalls and Debugging Tips	31
8.4	8.4 Real-World Applications	32
8.5	8.5 Closing Remarks	32

Chapter 1

Introduction to Graph Theory and the Shortest Path Problem

1.1 1.1 What is a Graph?

A **graph** $G = (V, E)$ is a mathematical structure consisting of a set of **vertices** (or nodes) V and a set of **edges** E that connect pairs of vertices. Graphs are the natural language for modeling networks: road systems, computer networks, social connections, dependency chains, and much more.

Formally:

- $V = \{v_1, v_2, \dots, v_n\}$ — the set of nodes.
- $E \subseteq V \times V$ — the set of edges, where each edge (u, v) represents a connection between vertex u and vertex v .

Graphs come in several important flavors, and the choice of shortest path algorithm depends heavily on which flavor you are dealing with.

1.1.1 1.1.1 Directed vs Undirected Graphs

- In a **directed graph** (digraph), each edge has a direction: an edge (u, v) allows travel from u to v but not necessarily from v to u .
- In an **undirected graph**, an edge (u, v) allows travel in both directions; it is equivalent to having both (u, v) and (v, u) .

1.1.2 1.1.2 Weighted vs Unweighted Graphs

- In an **unweighted graph**, every edge is considered to have the same “cost” (typically 1). Shortest path here means the path with the *fewest edges*.
- In a **weighted graph**, each edge (u, v) has an associated weight (or cost) $w(u, v)$, which could represent distance, time, money, or any other metric. Shortest path here means the path that *minimizes the total sum of edge weights*.

Weights can further be classified as:

- **Non-negative weights:** $w(u, v) \geq 0$ for all edges. This is the friendliest case, and allows greedy algorithms like Dijkstra's to work correctly.
- **Negative weights:** Some edges may have $w(u, v) < 0$. This complicates things significantly, since greedy strategies can fail, and we must worry about **negative cycles**.

1.1.3 1.1.3 Cyclic vs Acyclic Graphs

A graph that contains no cycles when directed is called a **Directed Acyclic Graph (DAG)**. DAGs admit a very fast, specialized shortest path algorithm based on topological sorting, discussed later in this guide.

1.2 1.2 What Does “Shortest Path” Mean?

Given a graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}$, and two vertices s (source) and t (target/destination), the **shortest path problem** asks: what is the path $P = (s = v_0, v_1, v_2, \dots, v_k = t)$ that minimizes

$$\text{cost}(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

over all possible paths from s to t ?

There are several important *variants* of this problem, and recognizing which variant you're facing is the single most important skill in this domain:

1. **Single-Pair Shortest Path:** Find the shortest path between one specific source s and one specific target t .
2. **Single-Source Shortest Path (SSSP):** Find the shortest path from one source s to *every other vertex* in the graph.
3. **Single-Destination Shortest Path:** Find the shortest path to a given destination from every other vertex (this is just SSSP on the reverse graph).
4. **All-Pairs Shortest Path (APSP):** Find the shortest path between *every* pair of vertices in the graph.

In practice, there is rarely an algorithm that solves single-pair shortest path faster than solving the full single-source problem (except with heuristics like A*), so most classical algorithms are designed around SSSP or APSP.

1.3 1.3 The Optimal Substructure Property

Nearly every shortest path algorithm relies on a deep and beautiful property: **optimal substructure**.

Claim: If $P = (v_0, v_1, \dots, v_k)$ is a shortest path from v_0 to v_k , then for any $0 \leq i \leq j \leq k$, the sub-path $P_{ij} = (v_i, v_{i+1}, \dots, v_j)$ is a shortest path from v_i to v_j .

Proof (by contradiction): Suppose the sub-path P_{ij} is *not* the shortest path from v_i to v_j . Then there exists some other path P'_{ij} from v_i to v_j with strictly smaller cost. We could then replace P_{ij} in P with P'_{ij} , producing a new path from v_0 to v_k with strictly smaller total cost than P . This contradicts the assumption that P is a shortest path. ■

This property is what allows us to build shortest paths incrementally — via dynamic programming (Bellman-Ford, Floyd-Warshall) or via greedy relaxation (Dijkstra) — rather than needing to enumerate every possible path (which would be exponential).

1.4 The Concept of “Relaxation”

Almost every shortest path algorithm in this guide is built from a single fundamental operation called **edge relaxation**.

We maintain a distance estimate $dist[v]$ for every vertex v , initialized to ∞ for all vertices except the source, where $dist[s] = 0$. The **relax** operation on edge (u, v) with weight $w(u, v)$ works as follows:

if $dist[u] + w(u, v) < dist[v]$: $dist[v] \leftarrow dist[u] + w(u, v)$, $parent[v] \leftarrow u$

In words: “If I can reach v more cheaply by going through u than by whatever path I currently know about, update my belief about the shortest distance to v .”

This simple idea, applied in the right order and the right number of times, is the beating heart of Dijkstra’s algorithm, Bellman-Ford, SPFA, and DAG shortest paths. The algorithms in this guide differ mainly in:

1. **The order** in which they relax edges.
2. **How many times** they need to relax each edge before the distance estimates are guaranteed correct.
3. **What guarantees** (invariants) they can offer about correctness at intermediate stages.

1.5 Graph Representations

Before diving into algorithms, it’s worth understanding how graphs are represented in code, since this directly affects algorithm complexity.

1.5.1 Adjacency Matrix

A 2D array $adj[n][n]$ where $adj[u][v]$ stores the weight of edge (u, v) (or infinity/0 if no edge exists).

- **Space:** $O(V^2)$
- **Edge lookup:** $O(1)$
- **Iterating over neighbors of a vertex:** $O(V)$

- Best suited for **dense graphs** and algorithms like Floyd-Warshall that inherently need $O(V^2)$ or $O(V^3)$ time anyway.

1.5.2 Adjacency List

An array of lists, where $\text{adj}[u]$ contains pairs (v, weight) for every edge leaving u .

- **Space:** $O(V + E)$
- **Iterating over neighbors of a vertex u :** $O(\deg(u))$
- Best suited for **sparse graphs**, which is the overwhelming majority of real-world graphs and competitive programming problems. Dijkstra, Bellman-Ford, BFS, and SPFA are typically implemented on adjacency lists.

1.5.3 Edge List

A flat list of triples (u, v, w) representing all edges. This representation is preferred specifically for **Bellman-Ford**, since that algorithm needs to iterate over *all edges* repeatedly, regardless of source vertex structure.

1.6 Notation Used Throughout This Guide

- V = number of vertices, E = number of edges.
- $\text{dist}[v]$ = current best known distance from source to v .
- $\pi[v]$ or $\text{parent}[v]$ = predecessor of v on the shortest path, used to reconstruct the actual path.
- ∞ = a sentinel value larger than any possible path cost (in code, typically `INT_MAX` or `1e18` for `long long`, chosen carefully to avoid overflow during addition).

Chapter 2

Breadth-First Search: Shortest Paths in Unweighted Graphs

2.1 2.1 Motivation

When every edge has the same weight (equivalently, an **unweighted graph**), the shortest path is simply the path with the fewest edges. It would be wasteful to use a general-purpose weighted algorithm like Dijkstra’s when a much simpler and faster technique exists: **Breadth-First Search (BFS)**.

2.2 2.2 Core Idea

BFS explores the graph **layer by layer**, radiating outward from the source. It first visits all vertices at distance 1 from the source, then all vertices at distance 2, then distance 3, and so on. This “wavefront” expansion is precisely what guarantees shortest paths in an unweighted setting.

The key data structure that enables this layer-by-layer expansion is a **queue** (First-In-First-Out). We enqueue the source, then repeatedly dequeue a vertex, look at its neighbors, and enqueue any neighbor we haven’t seen yet, recording its distance as one more than the current vertex’s distance.

2.3 2.3 Why BFS Guarantees Shortest Paths

Claim: When BFS dequeues a vertex v for the first time and assigns $dist[v] = d$, then d is indeed the length of the shortest path from s to v .

Proof sketch (by induction on distance layers):

- **Base case:** $dist[s] = 0$, trivially correct since the shortest path from s to itself has length 0.
- **Inductive step:** Assume BFS has correctly computed all shortest distances up to some value d . Because BFS processes vertices strictly in the order it enqueues

them (FIFO), and a vertex is enqueued *only* by being adjacent to some vertex at distance d , any vertex discovered while processing distance- d vertices must be at distance exactly $d + 1$ (it cannot be closer, since that would mean it should have already been discovered at a smaller distance under the inductive hypothesis; and it cannot be farther in the shortest-path sense, since we have just found a path of length $d + 1$ to it). Hence all vertices assigned distance $d + 1$ truly have shortest-path distance $d + 1$. ■

The invariant “each layer is fully processed before the next layer begins” is what makes the FIFO queue essential — using a stack (DFS) instead would break this guarantee entirely, since DFS can plunge deep into the graph before backing up, producing incorrect distance labels.

2.4 Multi-Source BFS

A very useful generalization: if you need the shortest distance from *any* of a set of source vertices $S = \{s_1, s_2, \dots, s_k\}$ to every other vertex, you can initialize the BFS queue with *all* sources simultaneously, each with $dist = 0$. The wavefronts from each source expand outward together, and the first time any vertex is reached, it’s guaranteed to be reached from whichever source is nearest to it. This is a common trick in problems like “rotting oranges,” “nearest exit in a maze,” or “distance to nearest guarded cell.”

2.5 0-1 BFS: A Bridge to Weighted Graphs

A special and very useful case arises when edge weights are restricted to only **0** or **1**. Ordinary BFS breaks down here because a 0-weight edge shouldn’t advance you to the “next layer” — but Dijkstra’s algorithm (with its $O(E \log V)$ priority queue) would be overkill.

The solution is **0-1 BFS**, which uses a **deque** (double-ended queue) instead of a plain queue:

- When relaxing an edge of weight **0**, push the neighbor to the **front** of the deque (it belongs in the same “layer” we’re currently processing).
- When relaxing an edge of weight **1**, push the neighbor to the **back** of the deque (it belongs to the next layer).

This maintains the invariant that the deque always contains at most two distinct distance values at any time, sorted in non-decreasing order from front to back — which lets us pop from the front and always process the globally smallest known tentative distance next, exactly like Dijkstra’s algorithm, but without needing a logarithmic-time heap. This gives $O(V + E)$ complexity, strictly better than Dijkstra’s $O(E \log V)$.

2.6 Complexity of BFS

- **Time:** $O(V + E)$ — every vertex is enqueued once, and every edge is examined once.

- **Space:** $O(V)$ for the distance array, visited markers, and the queue.

2.7 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Standard BFS shortest path in an unweighted graph.
// adj[u] contains all vertices v such that there is an edge u -> v.
vector<int> bfsShortestPath(int n, vector<vector<int>>& adj, int src) {
    vector<int> dist(n, -1);    // -1 means "unreachable"
    queue<int> q;

    dist[src] = 0;
    q.push(src);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {    // not visited yet
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist; // dist[v] = shortest number of edges from src to v
}

// 0-1 BFS: edge weights are only 0 or 1.
// adj[u] contains pairs {v, weight}.
vector<int> zeroOneBFS(int n, vector<vector<pair<int,int>>>& adj, int src) {
    const int INF = INT_MAX;
    vector<int> dist(n, INF);
    deque<int> dq;

    dist[src] = 0;
    dq.push_back(src);

    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
}
```

```
    }  
    }  
    return dist;  
}
```

Chapter 3

Dijkstra's Algorithm

3.1 Motivation

Once edges carry different non-negative weights, BFS no longer works — the path with fewer edges might actually have a larger total weight than a path with more, cheaper edges. We need an algorithm that accounts for edge weights while still exploring efficiently. **Dijkstra's Algorithm** (1959) is the classical greedy solution for single-source shortest paths when **all edge weights are non-negative**.

3.2 Core Idea

Dijkstra's algorithm maintains a set of vertices whose shortest distance from the source is **already finalized** and known to be correct. At each step, it selects the *unfinalized* vertex with the smallest tentative distance, finalizes it, and relaxes all of its outgoing edges.

The intuition is greedy: among all vertices not yet finalized, the one with the smallest tentative distance **cannot possibly be improved later**, because all edge weights are non-negative, so any path through a not-yet-visited (and therefore farther, or equally far) vertex could only be equal or longer.

3.3 Step-by-Step Procedure

1. Initialize $dist[s] = 0$ and $dist[v] = \infty$ for all $v \neq s$.
2. Insert all vertices into a priority queue (min-heap), keyed by $dist$.
3. While the priority queue is not empty:
 - a. Extract the vertex u with the smallest $dist[u]$ that has not yet been finalized.
 - b. Mark u as finalized.
 - c. For every neighbor v of u via edge weight $w(u, v)$: relax the edge — if $dist[u] + w(u, v) < dist[v]$, update $dist[v]$ and push v (with its new distance) into the priority queue.
4. When the queue is empty, every reachable vertex has its final shortest distance in $dist[\cdot]$.

3.4 Why Greedy Works: Proof of Correctness

Claim: When Dijkstra’s algorithm finalizes a vertex u (i.e., extracts it from the priority queue and marks it done), $dist[u]$ equals the true shortest distance from s to u .

Proof (by induction on the order of finalization):

Assume, inductively, that every vertex finalized *before* u has a correct final distance. We must show $dist[u]$ is also correct at the moment of finalization.

Suppose, for contradiction, that the true shortest path P from s to u has cost strictly less than $dist[u]$ at this moment. Path P must leave the set of already-finalized vertices at some point — let x be the last finalized vertex on P before it exits into un-finalized territory, and let y be the very next vertex on P after x (possibly $y = u$ itself).

Since x is finalized, $dist[x]$ is correct (by the inductive hypothesis) and the edge (x, y) has already been relaxed, so $dist[y] \leq dist[x] + w(x, y) = \text{cost of the prefix of } P \text{ up to } y$.

Because all subsequent edge weights on P (from y to u) are **non-negative**, the cost of the full path P is at least the cost of its prefix up to y , which is at least $dist[y]$.

So we have $dist[y] \leq \text{cost}(P) < dist[u]$.

But Dijkstra’s algorithm always extracts the *unfinalized* vertex with the **smallest** tentative distance — and y is unfinalized (it lies beyond x , the last finalized vertex on P) with $dist[y] < dist[u]$. This means y , not u , should have been extracted first. Contradiction. ■

This proof is precisely why Dijkstra’s algorithm **fails in the presence of negative edge weights**: the step “the cost of the full path is at least the cost of its prefix” relies entirely on all remaining edges being non-negative. If an edge later in the path were negative, a path could look expensive early on but become cheaper than a currently-finalized shorter-looking path — and greedy finalization could lock in a wrong answer permanently.

3.5 Data Structure Choice and Complexity

The efficiency of Dijkstra’s algorithm hinges on how quickly we can extract the minimum tentative distance and update distances.

Priority Queue Implementation	Time Complexity
Array / Linear scan (no heap)	$O(V^2)$
Binary heap (e.g. <code>std::priority_queue</code>)	$O((V + E) \log V)$
Fibonacci heap	$O(E + V \log V)$

In competitive programming and most practical software, the **binary heap** version is used, since Fibonacci heaps have poor constant factors despite better

asymptotic complexity. Note also the “**lazy deletion**” trick commonly used with `std::priority_queue` in C++: since the STL heap doesn’t support efficient decrease-key, we simply push a new (better) distance entry every time we relax an edge, and when popping, we skip any entry whose recorded distance no longer matches the current best known distance for that vertex (i.e., it’s a stale, already-superseded entry).

For very dense graphs (where $E \approx V^2$), the simple $O(V^2)$ array-based version can actually outperform the heap-based version, since it avoids the $\log V$ factor and the overhead of heap operations.

3.6 Path Reconstruction

To recover the actual shortest path (not just its length), maintain a `parent[v]` array updated every time you relax edge (u, v) successfully: `parent[v] = u`. After the algorithm finishes, walk backward from the destination using `parent[]` until you reach the source, then reverse the resulting list.

3.7 Handling Negative Weights: What Goes Wrong

If Dijkstra’s algorithm is run on a graph with a negative edge, it can finalize a vertex with an incorrect (too large) distance, and since finalized vertices are never revisited, this error can never be corrected. This is precisely why **Bellman-Ford** (Chapter 4) exists — it sacrifices speed for the ability to handle negative weights correctly.

3.8 Complexity Summary

- **Time:** $O((V + E) \log V)$ with a binary heap and adjacency list.
- **Space:** $O(V + E)$.
- **Restriction:** All edge weights must be non-negative.

3.9 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 1e18;

// adj[u] = list of {v, weight} pairs
vector<ll> dijkstra(int n, vector<vector<pair<int,ll>>>& adj, int src) {
    vector<ll> dist(n, INF);
    vector<int> parent(n, -1);
    // min-heap of {distance, vertex}
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
```

```
dist[src] = 0;
pq.push({0, src});

while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();

    if (d > dist[u]) continue; // stale/lazy-deleted entry, skip

    for (auto [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            parent[v] = u;
            pq.push({dist[v], v});
        }
    }
}
return dist;
}
```

Chapter 4

The Bellman-Ford Algorithm

4.1 4.1 Motivation

Dijkstra’s algorithm is fast, but it breaks down whenever a graph has negative edge weights, because its greedy “finalize-and-never-revisit” strategy assumes distances can only grow as we consider more edges along a path. The **Bellman-Ford algorithm** (Bellman 1958, Ford 1956) takes a different, more patient approach based on **dynamic programming**, which correctly handles negative edges and can even **detect negative cycles**.

4.2 4.2 Core Idea

Bellman-Ford relies on a simple but powerful fact: in a graph with V vertices, any *simple* shortest path (one that doesn’t revisit a vertex) can contain **at most** $V - 1$ **edges**. This is because a simple path visits each vertex at most once, so a path through all V vertices has exactly $V - 1$ edges, and no simple path can be longer.

The algorithm therefore relaxes **every single edge in the graph**, and repeats this process $V - 1$ **times**. After k rounds of relaxing all edges, the algorithm is guaranteed to have found the correct shortest distance for any vertex whose shortest path uses at most k edges.

4.3 4.3 Step-by-Step Procedure

1. Initialize $dist[s] = 0$ and $dist[v] = \infty$ for all other vertices.
2. Repeat the following $V - 1$ **times**: for every edge (u, v) with weight w in the graph, relax it — if $dist[u] + w < dist[v]$, set $dist[v] = dist[u] + w$.
3. (Optional but crucial) Do **one more pass** over all edges. If any edge can still be relaxed (i.e., $dist[u] + w < dist[v]$), this proves the graph contains a **negative-weight cycle** reachable from the source, and shortest paths are not well-defined (they can be made arbitrarily small by looping around the cycle indefinitely).

4.4 4.4 Why $V - 1$ Iterations Suffice: Proof of Correctness

Claim: After the k -th iteration of relaxing all edges, $dist[v]$ is correct for every vertex v whose shortest path from s uses **at most k edges**.

Proof (by induction on k):

- **Base case ($k = 0$):** Before any relaxation, $dist[s] = 0$ is correct (the shortest path with 0 edges, from s to itself), and all others are ∞ , vacuously “correct” for paths using 0 edges (no such path exists for $v \neq s$).
- **Inductive step:** Assume after k iterations, $dist[v]$ is correct for all vertices whose shortest path uses at most k edges. Consider a vertex u whose shortest path P from s uses **exactly $k + 1$ edges**. Let x be the second-to-last vertex on P , so P is (shortest path to x using k edges) followed by edge (x, u) . By the inductive hypothesis, $dist[x]$ is already correct after k iterations. During the $(k + 1)$ -th iteration, we relax **every edge**, including (x, u) — so $dist[u]$ becomes at most $dist[x] + w(x, u)$, which equals the cost of P , the true shortest distance. Since $dist[u]$ can never go below the true shortest distance (relaxation only ever tightens an upper bound, never produces something smaller than truth), $dist[u]$ is now exactly correct. ■

Since any *simple* shortest path has at most $V - 1$ edges, running $V - 1$ full iterations guarantees correctness for all vertices, provided there is no negative cycle.

4.5 4.5 Negative Cycle Detection

If, after $V - 1$ iterations, a V -th pass still finds an edge (u, v) that can be relaxed further, this is proof that some shortest path would need more than $V - 1$ edges — which is only possible if the path revisits vertices, i.e., loops around a cycle. Since it’s still improving, that cycle must have **negative total weight**. This is the standard technique for negative cycle detection, widely used in arbitrage-detection problems in currency exchange graphs, where a “cycle” of trades that ends up with more money than you started represents a negative cycle under a $-\log$ transformation of exchange rates.

To identify *which* vertices are affected by the negative cycle (not just confirm its existence), a common technique is to run one more pass (the V -th) and mark any vertex that still relaxes; then run a BFS/DFS from all such marked vertices to flag every vertex reachable from the cycle, since their distances are effectively $-\infty$ (undefined).

4.6 4.6 SPFA: An Optimization (with a Caveat)

The **Shortest Path Faster Algorithm (SPFA)** is a popular queue-based optimization of Bellman-Ford. Instead of blindly relaxing all E edges in every one of the $V - 1$ rounds, SPFA only re-examines edges leaving a vertex **whose own distance was just improved**. This is implemented with a FIFO queue: whenever $dist[v]$ is successfully updated, v is pushed onto the queue (if not already present), so its outgoing edges get relaxed soon.

In practice, on many graphs SPFA is much faster than the naive $O(VE)$ Bellman-Ford, behaving close to $O(E)$ on average. However, its **worst-case complexity is still $O(VE)$** , and on specially adversarial or dense graphs, it can be slow — competitive programmers sometimes deliberately construct “anti-SPFA” test cases. It is a useful practical tool, but it should not be relied upon as an asymptotic improvement.

4.7 4.7 Complexity Summary

- **Time:** $O(V \cdot E)$ for standard Bellman-Ford.
- **Space:** $O(V)$ for distances (plus $O(E)$ if edges are stored as a list).
- **Advantage over Dijkstra:** works correctly with negative edge weights, and can detect negative cycles.
- **Disadvantage:** significantly slower than Dijkstra when weights are all non-negative.

4.8 4.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 1e18;

struct Edge { int u, v; ll w; };

// Returns {dist array, hasNegativeCycle}
pair<vector<ll>, bool> bellmanFord(int n, vector<Edge>& edges, int src) {
    vector<ll> dist(n, INF);
    dist[src] = 0;

    // Relax all edges V-1 times
    for (int i = 0; i < n - 1; i++) {
        for (auto& e : edges) {
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
            }
        }
    }

    // One more pass to detect negative cycles
    bool hasNegCycle = false;
    for (auto& e : edges) {
        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
            hasNegCycle = true;
            break;
        }
    }
}
```

```
    return {dist, hasNegCycle};  
}
```

Chapter 5

Shortest Paths in Directed Acyclic Graphs (DAGs)

5.1 5.1 Motivation

If we know in advance that a graph is a **Directed Acyclic Graph (DAG)** — meaning it's directed and has no cycles — we can compute single-source shortest paths in **linear time**, $O(V + E)$, even in the presence of **negative edge weights**. This is faster than Dijkstra and doesn't share Dijkstra's restriction to non-negative weights, precisely because the absence of cycles sidesteps the entire issue of needing to “wait and see” whether a cheaper path might still emerge.

5.2 5.2 Core Idea: Topological Order

A **topological order** of a DAG is a linear ordering of its vertices such that for every directed edge (u, v) , u appears before v in the ordering. Such an ordering always exists for a DAG (and is generally not unique).

The key insight: if we process vertices **in topological order**, then by the time we reach any vertex u , **all** vertices that could contribute to u 's shortest distance (i.e., all predecessors of u on any path from the source) have *already been fully processed*, and $dist[u]$ is already finalized and correct. We can then safely relax all of u 's outgoing edges once, and never need to revisit u again.

5.3 5.3 Step-by-Step Procedure

1. Compute a topological ordering of the DAG (via DFS-based ordering or Kahn's algorithm using in-degrees).
2. Initialize $dist[s] = 0$, and $dist[v] = \infty$ for all $v \neq s$.
3. Process vertices **in topological order**. For each vertex u (in order), relax every outgoing edge (u, v) : if $dist[u] + w(u, v) < dist[v]$, update $dist[v]$.
4. After processing all vertices, $dist[\cdot]$ contains the correct shortest distances.

5.4 Proof of Correctness

Claim: Processing vertices in topological order and relaxing each vertex's outgoing edges exactly once produces correct shortest distances for all vertices.

Proof: By definition of topological order, every edge (x, y) satisfies “ x comes before y ”. Consider any vertex v and its true shortest path $P = (s = u_0, u_1, \dots, u_k = v)$. Every vertex on P appears in increasing topological order (since every consecutive pair is connected by an edge that respects the ordering). By induction on the position in this path: $\text{dist}[u_0] = 0$ is correct. If $\text{dist}[u_i]$ is correct when the algorithm processes u_i , then when the algorithm reaches u_i in topological order, it relaxes edge (u_i, u_{i+1}) , and since u_i is processed strictly before u_{i+1} , this relaxation happens before u_{i+1} 's own distance is “closed off” or needed elsewhere — guaranteeing $\text{dist}[u_{i+1}] \leq \text{dist}[u_i] + w(u_i, u_{i+1})$, which is exactly the shortest-path recurrence. Since relaxation never underestimates, equality holds by induction, all the way to $u_k = v$. ■

Because the topological order guarantees each vertex is only ever “finished” after all of its possible predecessors have contributed their relaxations, we never need to revisit a vertex — unlike Bellman-Ford, which needs up to $V - 1$ full passes precisely because it has no such guaranteed order to exploit.

5.5 Why This Handles Negative Weights Just Fine

Dijkstra's proof of correctness depended on the *greedy* claim that the globally smallest tentative distance can never be improved later — a claim that breaks with negative weights. The DAG shortest path algorithm makes no such greedy claim at all. It simply guarantees, via the topological order, that all relevant predecessors are processed first. Negative weights don't threaten this guarantee in any way, since the ordering is purely structural (based on edge direction), not based on weight values.

5.6 Applications

- **Longest path in a DAG:** Simply negate all edge weights (multiply by -1) and run the same shortest-path algorithm; the “shortest path” in the negated graph is the longest path in the original. This trick only works because the DAG algorithm tolerates negative weights — it would not work with Dijkstra.
- **Critical path method (CPM) / project scheduling:** modeling tasks as nodes and dependencies as edges, the longest path through the DAG determines the minimum time to complete a project.
- **Dynamic programming problems:** many classic DP problems (e.g., longest increasing subsequence) can be reframed as DAG shortest/longest path problems, since DP dependency graphs are inherently acyclic.

5.7 Complexity Summary

- **Time:** $O(V + E)$ — a topological sort ($O(V + E)$) followed by one relaxation pass over all edges ($O(E)$).
- **Space:** $O(V + E)$.
- **Restriction:** Graph must be a DAG (directed, no cycles). Works with negative weights.

5.8 C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 1e18;

void topoSort(int u, vector<vector<pair<int,ll>>>& adj,
              vector<bool>& visited, stack<int>& order) {
    visited[u] = true;
    for (auto [v, w] : adj[u])
        if (!visited[v]) topoSort(v, adj, visited, order);
    order.push(u); // push after visiting all descendants
}

vector<ll> dagShortestPath(int n, vector<vector<pair<int,ll>>>& adj, int src) {
    vector<bool> visited(n, false);
    stack<int> order;

    for (int i = 0; i < n; i++)
        if (!visited[i]) topoSort(i, adj, visited, order);

    vector<ll> dist(n, INF);
    dist[src] = 0;

    while (!order.empty()) {
        int u = order.top(); order.pop();
        if (dist[u] == INF) continue; // unreachable from src
        for (auto [v, w] : adj[u])
            if (dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
    }
    return dist;
}
```

Chapter 6

Floyd-Warshall: All-Pairs Shortest Paths

6.1 6.1 Motivation

So far, every algorithm we’ve discussed solves the **single-source** shortest path problem. But what if we need the shortest distance between **every pair** of vertices? Running Dijkstra or Bellman-Ford from every single vertex would work, but the **Floyd-Warshall algorithm** offers an elegant dynamic-programming alternative that’s simple to implement and works with negative edges (as long as there’s no negative cycle).

6.2 6.2 Core Idea: Intermediate Vertices

Floyd-Warshall is built around a beautifully simple dynamic programming state:

$dist_k[i][j]$ = the shortest distance from i to j using only vertices $\{1, 2, \dots, k\}$ as allowed intermediates

The recurrence is based on a single yes/no decision: **does the optimal path from i to j (using only $\{1, \dots, k\}$ as intermediates) pass through vertex k or not?**

$$dist_k[i][j] = \min \left(dist_{k-1}[i][j], dist_{k-1}[i][k] + dist_{k-1}[k][j] \right)$$

- If the best path doesn’t use k at all, the answer is unchanged from the previous step: $dist_{k-1}[i][j]$.
- If the best path does use k , then it must go from i to k and then from k to j , and by optimal substructure, both of those sub-paths must themselves be shortest paths using only intermediates from $\{1, \dots, k-1\}$ (since k itself, as an endpoint of the join, is used exactly once as a “hinge”).

We iterate k from 1 to V , and after considering all V possible intermediate vertices, $dist_V[i][j]$ gives the true shortest distance from i to j using any vertices as intermediates — which is exactly what we want.

6.3 6.3 The Remarkable In-Place Simplification

In practice, we don't need to keep V separate 2D arrays for each value of k . We can use a **single 2D array** $\text{dist}[i][j]$ and update it in place as k increases, because when computing $\text{dist}_k[i][j]$, the values $\text{dist}_{k-1}[i][k]$ and $\text{dist}_{k-1}[k][j]$ are never themselves updated during the k -th round (a path from i to k or from k to j that uses k itself as an intermediate would need to visit k twice, which is never beneficial in a graph with no negative cycles). This allows the elegant three-nested-loop implementation that makes Floyd-Warshall so famous for its simplicity.

6.4 6.4 Step-by-Step Procedure

1. Initialize $\text{dist}[i][j] = w(i,j)$ if edge (i,j) exists, $\text{dist}[i][i] = 0$, and $\text{dist}[i][j] = \infty$ otherwise.
2. For $k = 1$ to V : For $i = 1$ to V : For $j = 1$ to V : $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$
3. After the loops finish, $\text{dist}[i][j]$ holds the shortest distance between every pair.

6.5 6.5 Negative Cycle Detection

After running the algorithm, check the diagonal: if $\text{dist}[i][i] < 0$ for any vertex i , then i lies on (or can reach and return from) a negative cycle, and shortest paths through i are not well-defined.

6.6 6.6 Complexity Summary

- **Time:** $O(V^3)$ — three nested loops over all vertices.
- **Space:** $O(V^2)$ for the distance matrix.
- **When to use:** Best suited for **dense graphs** or when V is small (roughly $V \leq 400$ –500 in competitive programming, given the V^3 cost), and especially when you genuinely need **all-pairs** distances, not just from a single source. For sparse graphs, running Dijkstra from every vertex — $O(V \cdot E \log V)$ — is typically much faster when $E \ll V^2$.

6.7 6.7 Johnson's Algorithm: A Faster Alternative for Sparse Graphs with Negative Edges

When a graph is **sparse** but still has negative edges (though no negative cycle), Floyd-Warshall's $O(V^3)$ can be wasteful. **Johnson's Algorithm** provides a clever way to still use the fast Dijkstra's algorithm (which requires non-negative weights) on every vertex, even in the presence of negative edges, by first **reweighting** the graph.

6.7.1 6.7.1 The Reweighting Trick

1. Add a new virtual vertex q connected to every other vertex with a zero-weight edge.
2. Run Bellman-Ford from q to compute $h[v]$ = shortest distance from q to every vertex v (this also detects negative cycles in the original graph, and this step is the only place we still need Bellman-Ford's negative-weight tolerance).
3. Reweight every edge: $w'(u, v) = w(u, v) + h[u] - h[v]$.

Claim: The reweighted graph has **all non-negative edge weights**, and shortest paths in the reweighted graph correspond exactly to shortest paths in the original graph (just shifted by a constant depending only on the endpoints).

Why the reweighted weights are non-negative: Because $h[v]$ is the shortest distance from q to v , the triangle inequality tells us $h[v] \leq h[u] + w(u, v)$ for every edge (u, v) (otherwise the edge itself would offer an improving relaxation, contradicting that h is already a shortest-distance array). Rearranging: $w(u, v) + h[u] - h[v] \geq 0$, exactly the definition of $w'(u, v)$.

Why shortest paths are preserved: For any path P from s to t , the reweighted cost telescopes:

$$\text{cost}'(P) = \sum (w(u, v) + h[u] - h[v]) = \text{cost}(P) + h[s] - h[t]$$

Since $h[s] - h[t]$ is a constant that doesn't depend on which path P we chose, minimizing $\text{cost}'(P)$ over all paths from s to t is exactly equivalent to minimizing $\text{cost}(P)$.

4. Run Dijkstra's algorithm from every vertex on the reweighted (now all non-negative) graph.
5. Convert distances back: the true shortest distance from u to v is $\text{dist}'(u, v) - h[u] + h[v]$.

6.7.2 6.7.2 Complexity of Johnson's Algorithm

- **Time:** $O(V \cdot E \log V)$ — one Bellman-Ford run ($O(VE)$) plus V runs of Dijkstra ($O(V \cdot E \log V)$).
- This beats Floyd-Warshall's $O(V^3)$ whenever the graph is sparse, i.e., $E = O(V^2 / \log V)$ or smaller.

6.8 6.8 C++ Implementation of Floyd-Warshall

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 1e18;

// dist[i][j] initialized with edge weights beforehand (INF if no edge, 0 on diagonal)
void floydWarshall(int n, vector<vector<ll>>& dist) {
```

```
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        if (dist[i][k] == INF) continue;           // small optimization
        for (int j = 0; j < n; j++) {
            if (dist[k][j] == INF) continue;
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}
// Negative cycle check: if dist[i][i] < 0, vertex i is on/reaches a negative cycle
}
```

Chapter 7

The A* Algorithm: Informed Search

7.1 7.1 Motivation

Dijkstra’s algorithm is “uninformed” — it expands vertices purely based on their distance from the source, with no notion of *where the target actually is*. If you only care about the shortest path from s to one specific target t (single-pair shortest path), and you have some extra knowledge about the geometry or structure of the problem (for instance, straight-line distance on a map), you can guide the search to explore far more efficiently. This is the idea behind **A* search**, one of the most important algorithms in AI and pathfinding.

7.2 7.2 Core Idea: Heuristic-Guided Priority

A* is essentially Dijkstra’s algorithm, but instead of prioritizing vertices purely by $\text{dist}[v]$ (the cost so far, often denoted $g(v)$), it prioritizes them by

$$f(v) = g(v) + h(v)$$

where:

- $g(v)$ = the actual shortest distance found so far from s to v (exactly like Dijkstra’s $\text{dist}[v]$).
- $h(v)$ = a **heuristic estimate** of the remaining distance from v to the target t .

Intuitively, $f(v)$ estimates the *total* cost of a path from s to t that passes through v : known cost so far, plus an educated guess of what remains. By always expanding the vertex with the smallest f -value, A* focuses its search effort in the direction most likely to lead toward the target, rather than blindly expanding in every direction.

7.3 The Admissibility Condition

A* is only guaranteed to find the true shortest path if the heuristic h is **admissible**: it must **never overestimate** the true remaining distance to the target. Formally, $h(v) \leq d^*(v, t)$ for every vertex v , where $d^*(v, t)$ is the true shortest distance from v to t .

Common admissible heuristics:

- **Euclidean (straight-line) distance**, for graphs embedded in physical space where edge weights represent real-world distances (a straight line is always the shortest possible path, so it never overestimates).
- **Manhattan distance**, for grid graphs where movement is restricted to four (or eight) directions.
- **The zero heuristic**, $h(v) = 0$ for all v — this is always trivially admissible, and reduces A* exactly to Dijkstra’s algorithm. This shows that A* is a strict generalization of Dijkstra.

7.4 Why Admissibility Guarantees Correctness

The proof of A*’s correctness closely mirrors Dijkstra’s proof, with one adjustment: instead of arguing that the smallest $g(v)$ can never be beaten, we argue that the smallest $f(v) = g(v) + h(v)$ among unfinalized vertices, when h never overestimates, still cannot be beaten by any path through an unfinalized vertex, because any such path’s true total cost is at least $g(v) + h(v)$ evaluated at the point it currently exits the finalized region — and since h underestimates, the true cost can never be smaller than what’s already been computed for the finalized target-optimal candidate. The full inductive proof follows the same skeleton as Dijkstra’s proof of correctness (Section 3.4), replacing “*dist*” with “*f*” and inserting the underestimate guarantee wherever the non-negativity of edge weights was used.

7.5 Consistency (Monotonicity): A Stronger and More Practical Condition

A slightly stronger condition than admissibility, called **consistency** (or monotonicity), is what’s typically required in practice for an efficient, correct implementation without needing to reopen already-finalized vertices:

$$h(u) \leq w(u, v) + h(v) \quad \text{for every edge } (u, v)$$

This is a “local” triangle-inequality-style condition. Every consistent heuristic is also admissible, but not vice versa. When h is consistent, f -values along any path are non-decreasing, which means A* behaves exactly like Dijkstra’s algorithm on a reweighted graph (compare to Johnson’s reweighting trick in Section 6.7) — once a vertex is finalized, it never needs to be revisited, keeping the algorithm efficient.

7.6 7.6 A* vs Dijkstra: A Summary Comparison

Aspect	Dijkstra	A*
Priority key	$g(v)$	$g(v) + h(v)$
Uses target information	No	Yes
Best for	Single-source (all destinations)	Single-pair (one specific destination)
Requires	Non-negative weights	Non-negative weights + admissible/consistent heuristic
Exploration pattern	Expands uniformly outward	Expands preferentially toward target

7.7 7.7 Applications

A* is the backbone of pathfinding in video games, robotics motion planning, GPS navigation systems, and puzzle-solving (like the 15-puzzle, using heuristics such as the number of misplaced tiles or the sum of Manhattan distances).

7.8 7.8 Complexity Summary

- **Time:** Worst case identical to Dijkstra, $O((V + E) \log V)$, but in practice far fewer vertices are actually expanded when the heuristic is informative, often making it dramatically faster in real-world graphs.
- **Space:** $O(V)$, same as Dijkstra, for the open/closed sets and priority queue.

7.9 7.9 C++ Implementation (Grid Example with Manhattan Heuristic)

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = 1e18;

// Example: A* on a 2D grid, moving in 4 directions with unit cost.
struct Cell { int r, c; };

int manhattan(Cell a, Cell b) {
    return abs(a.r - b.r) + abs(a.c - b.c);
}

int aStarGrid(vector<vector<int>>& grid, Cell start, Cell goal) {
    int rows = grid.size(), cols = grid[0].size();
```

```
vector<vector<int>> g(rows, vector<int>(cols, INT_MAX));
int dr[] = {0, 0, 1, -1}, dc[] = {1, -1, 0, 0};

// priority queue keyed on f = g + h
priority_queue<tuple<int,int,int>, vector<tuple<int,int,int>>, greater<>> pq;
g[start.r][start.c] = 0;
pq.push({manhattan(start, goal), start.r, start.c});

while (!pq.empty()) {
    auto [f, r, c] = pq.top(); pq.pop();
    if (r == goal.r && c == goal.c) return g[r][c]; // reached target

    if (f > g[r][c] + manhattan({r,c}, goal)) continue; // stale entry

    for (int i = 0; i < 4; i++) {
        int nr = r + dr[i], nc = c + dc[i];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
        if (grid[nr][nc] == 1) continue; // obstacle

        int newG = g[r][c] + 1;
        if (newG < g[nr][nc]) {
            g[nr][nc] = newG;
            int newF = newG + manhattan({nr, nc}, goal);
            pq.push({newF, nr, nc});
        }
    }
}
return -1; // unreachable
}
```

Chapter 8

Comparative Summary and Decision Guide

8.1 8.1 Choosing the Right Algorithm

The single most important skill in shortest-path problem solving is quickly identifying **which algorithm the problem structure demands**. Ask yourself these questions, in order:

1. **Are all edge weights equal (or is the graph unweighted)?** → Use **BFS** ($O(V + E)$). If weights are restricted to $\{0, 1\}$, use **0-1 BFS**.
2. **Do I need distances from many/all sources at once, in an unweighted or 0/1-weighted graph?** → Use **Multi-source BFS**.
3. **Is the graph a DAG (no cycles)?** → Use **DAG shortest path via topological sort** ($O(V + E)$), which also tolerates negative weights.
4. **Are all edge weights non-negative, and I need single-source shortest paths?** → Use **Dijkstra's Algorithm** ($O((V + E) \log V)$).
5. **Do I additionally know a good heuristic estimate of remaining distance to one specific target?** → Use **A*** for faster practical performance on the same non-negative-weight assumption.
6. **Might there be negative edge weights, and I need single-source shortest paths (or must detect negative cycles)?** → Use **Bellman-Ford** ($O(VE)$), or its practical speed-up **SPFA**.
7. **Do I need shortest distances between *all* pairs of vertices?**
 - If the graph is small/dense, or V is modest (roughly ≤ 500): use **Floyd-Warshall** ($O(V^3)$).
 - If the graph is large and sparse, and might have negative edges (but no negative cycle): use **Johnson's Algorithm** ($O(VE \log V)$), which reweights the graph and then runs Dijkstra from every vertex.

8.2 8.2 Master Comparison Table

Algorithm	Problem Type	Handles Negative Weights?	Detects Negative Cycles?	Time Complexity	Space
BFS	SSSP, unweighted	N/A	N/A	$O(V + E)$	$O(V)$
0-1 BFS	SSSP, weights in $\{0, 1\}$	N/A	N/A	$O(V + E)$	$O(V)$
DAG Shortest Path	SSSP, acyclic graphs	Yes	N/A (no cycles exist)	$O(V + E)$	$O(V + E)$
Dijkstra (heap)	SSSP, non-negative weights	No	No	$O((V + E) \log V)$	$O(V + E)$
Dijkstra (array)	SSSP, dense non-negative	No	No	$O(V^2)$	$O(V^2)$
A*	Single-pair, non-negative + heuristic	No	No	$O((V + E) \log V)$ worst case	$O(V)$
Bellman-Ford	SSSP, general weights	Yes	Yes	$O(VE)$	$O(V)$
SPFA	SSSP, general weights (practical)	Yes	Yes (with care)	$O(VE)$ worst, often faster	$O(V)$
Floyd-Warshall	APSP, general weights	Yes	Yes	$O(V^3)$	$O(V^2)$
Johnson's Algorithm	APSP, sparse + general weights	Yes	Yes	$O(VE \log V)$	$O(V^2)$

8.3 Common Pitfalls and Debugging Tips

- **Integer overflow:** When summing weights, $\text{dist}[u] + w$ can overflow if $\text{dist}[u]$ is already the sentinel “infinity” value and w is added to it carelessly. Always guard with a check like `if (dist[u] != INF)` before relaxing, or choose a sentinel small enough that adding a reasonable edge weight to it still doesn't overflow the integer type.
- **Using `int` instead of `long long`:** Whenever weights or the number of vertices/edges are large, shortest-path sums can exceed the range of a 32-bit in-

teger. Prefer `long long` (or `int64_t`) for distance arrays unless you are certain of the bounds.

- **Forgetting that Dijkstra fails on negative weights:** This is the single most common conceptual bug. If a problem statement mentions negative weights or costs that could be negative (e.g., profit, refunds, discounts), Dijkstra is the wrong tool — reach for Bellman-Ford, SPFA, or the DAG algorithm if applicable.
- **Stale priority queue entries:** Because `std::priority_queue` has no efficient decrease-key operation, always check whether a popped entry's distance matches the currently recorded best distance before processing it — otherwise you may re-process a vertex with outdated (larger) distance information.
- **Disconnected graphs:** Always initialize unreachable distances to infinity, and make sure your final answer explicitly handles vertices that remain at infinity (they were never reached).
- **Path reconstruction:** If you need the actual path (not just its length), remember to maintain and correctly update a `parent[]/prev[]` array during every successful relaxation, and reconstruct by walking backward from target to source, then reversing.

8.4 8.4 Real-World Applications

- **GPS and road navigation systems:** Dijkstra and A* (often combined with hierarchical/contraction techniques for continental-scale maps) compute driving directions.
- **Network routing protocols:** Link-state routing protocols like OSPF use Dijkstra's algorithm to compute optimal routes; distance-vector protocols like RIP use a Bellman-Ford-like distributed relaxation approach.
- **Currency arbitrage detection:** Modeling exchange rates as a graph with edge weight $-\log(\text{rate})$ turns "profitable arbitrage loop" into "negative cycle," detectable via Bellman-Ford.
- **Game AI and robotics path planning:** A* and its many variants (Dijkstra-based flow fields, Jump Point Search) are the standard for real-time navigation.
- **Airline and logistics scheduling:** Shortest/cheapest path problems underlie flight connection search engines and delivery route optimization.
- **Project management:** DAG-based longest-path computations power the Critical Path Method used in project scheduling software.

8.5 8.5 Closing Remarks

Every algorithm in this guide is, at its heart, a variation on the same core theme: maintain a distance estimate for each vertex, and repeatedly apply the **relaxation** operation until no more improvements are possible. What differs between algorithms is only the **order** and **number of times** relaxation is applied, driven by what structural guarantees the graph provides (unweighted, non-negative weights, acyclic, or fully general). Recognizing which structural guarantee your problem satisfies is the key that unlocks the fastest correct algorithm every time.